Architecture

Task 4. The short version: a modular monolith with a single-page client and realtime push, deployed as one container next to PostgreSQL.

1. What drives the design

- Every party is a small, strongly consistent world: two people adding the same song or skipping at the same time must not break the queue.
- Playback needs a 1-second ticker and long-lived WebSocket connections.
- The team is one or two people and the budget is zero.
- Guests must be able to open a link and join, so the API also serves the web client from the same origin.

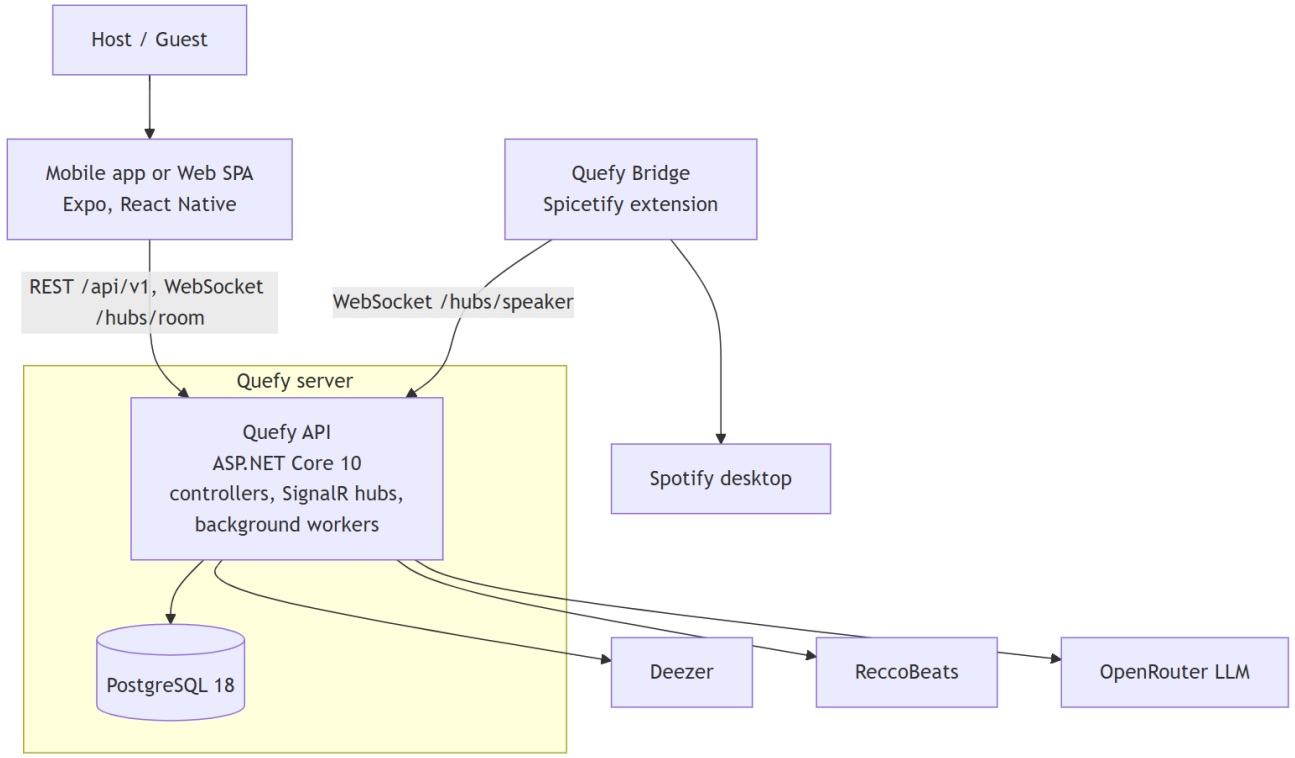
2. Style choice

Scores from 1 (bad) to 5 (good), weighted by how much each criterion matters to Quefy.

Criterion (weight)	Microservices	Serverless	Modular monolith
Per-room consistency (5)	2	2	5
Realtime and playback ticker (5)	4	2	5
Small team, short timeline (4)	1	3	5
Cost (4)	2	4	5
Scalability (3)	5	5	4
Modifiability (3)	5	3	4
Zero-install guests (4)	4	4	5
Total	88	88	**134**

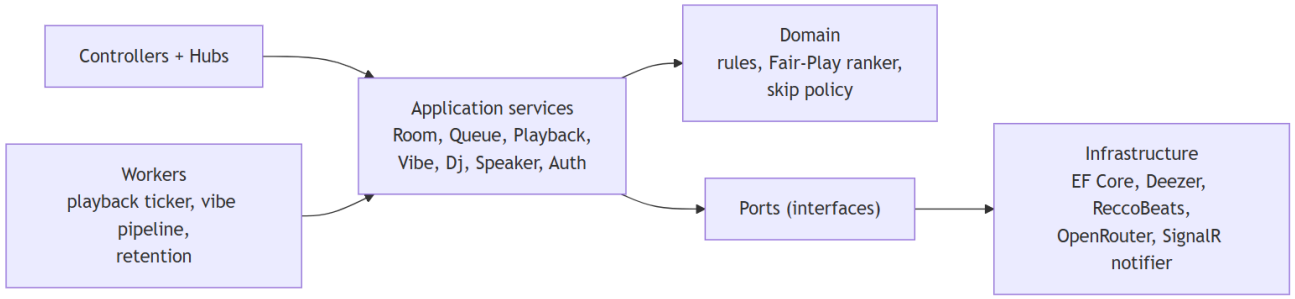
Decision: modular monolith (Clean Architecture: Api, Application, Domain, Infrastructure). The web client is a SPA. Microservices and serverless are not needed now; the modules (rooms, queue, playback, vibe/AI, catalog, speakers, identity) are separated, so the AI pipeline could become its own service later.

3. Containers



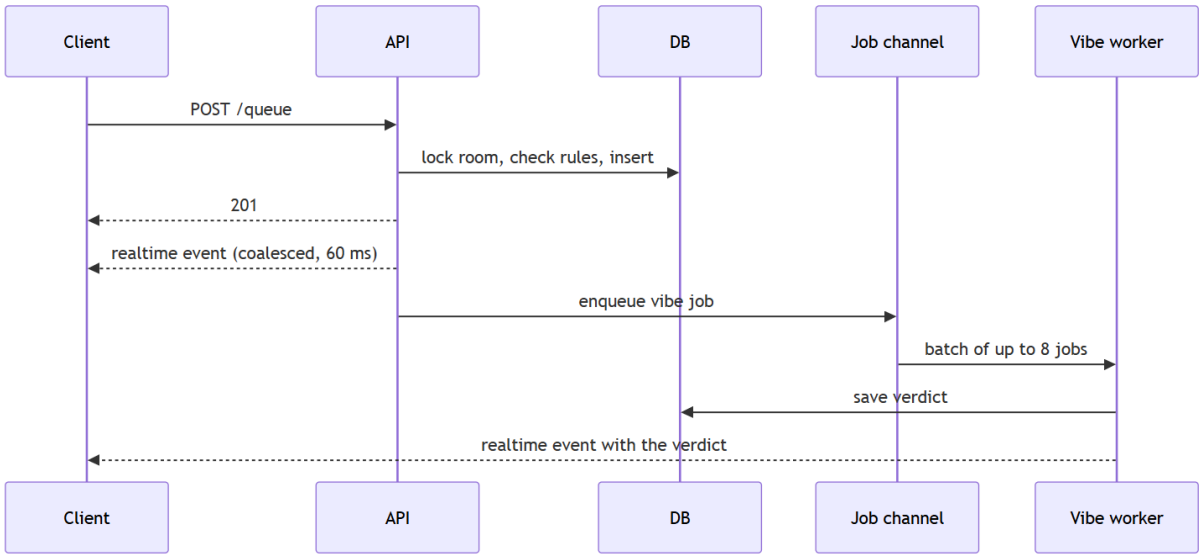
Container	Technology	Job
Mobile app / Web SPA	Expo SDK 57, expo-router, TanStack Query, Zustand	UI, optimistic votes, QR scan, preview speaker
Quefy API	ASP.NET Core 10, SignalR, EF Core	Business rules, auth, realtime, playback, AI, serves the web build
PostgreSQL	18	State, constraints that protect invariants, event log
Quefy Bridge	TypeScript, Spicetify	Plays tracks in Spotify and reports state every 3 s

4. Components inside the API



Dependencies only point inwards: Api, Infrastructure, Application, Domain. External services are behind interfaces so they can be faked in tests.

5. Add a track (runtime view)



6. Cross-cutting parts

- **Errors:** every error is an RFC 9457 problem document with a stable `code` (document 05).
- **Realtime:** the server sends the whole changed part with a room version number; clients ignore older versions and take a full snapshot after reconnecting.
- **Resilience:** timeouts, retries and a circuit breaker on every outgoing HTTP client (Polly).
- **Observability:** OpenTelemetry metrics and traces, `/health/live` and `/health/ready`.
- **Configuration:** from environment and secrets; nothing secret in git.

7. Key decisions

#	Decision	Why
1	Modular monolith with Clean Architecture	Best fit for consistency, team size and cost
2	The server decides what plays; speakers only obey	One source of truth, speakers can be swapped
3	Spotify is controlled through a Spicetify extension	Spotify's Web API player needs Premium; this works without it (demo only)
4	Deezer for the catalog, ReccoBeats for audio features	Free and no user login needed
5	Fair-Play is a formula, not an LLM	Predictable and explainable; the LLM only judges the vibe
6	Per-room lock by bumping a version column	Simple serialization with plain PostgreSQL
7	Realtime events carry the full changed part with a version	Lost or reordered events heal themselves
8	Anonymous device identity with rotating refresh tokens	No sign-up friction
9	Free LLM through OpenRouter with fallbacks	Zero budget; AI is optional for the party to work

#	Decision	Why
10	The API serves the web build	One URL, no CORS

8. Growing later

One API node handles a few dozen parties. To scale out: add a Redis backplane for SignalR, move in-memory limits and presence to Redis, run several containers behind a load balancer with sticky sessions, and add a read replica for analytics. The deployment target is in document 10.

9. Stack

Layer	Technology
Server	.NET 10, ASP.NET Core, SignalR, EF Core, Polly, OpenTelemetry
Database	PostgreSQL 18
Client	Expo SDK 57, React Native, NativeWind, TanStack Query, Zustand
Bridge	TypeScript, Spicetify
AI	OpenRouter (free model)
Tests	xUnit, Node end-to-end scripts